DeepSpill – Oil Spill Detection

BY AKSHAT RAIKAR



Problem Statement

IMPACT OF OIL SPILLS

Oil spills are devastating, harming **marine ecosystems** and local economies. Current manual monitoring methods are **inefficient** and require significant resources. There is an urgent need for an **automated** solution that efficiently detects oil spills in real-time, ensuring swift responses.

Project Objectives

01 DETECTION

Develop a deep learning model that **accurately identifies** oil spills from satellite imagery, enhancing the speed and reliability of environmental assessments compared to traditional methods.

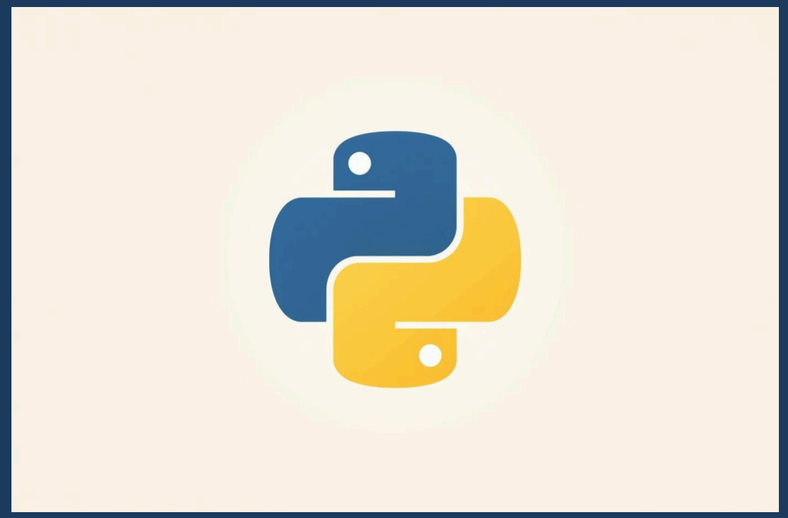
02 AUTOMATION

Automate the image classification process to facilitate **rapid monitoring** of oil spills, allowing for timely interventions and minimizing the impact on marine ecosystems and coastal economies.

03 SUPPORT

Provide crucial support to authorities by enabling **early detection** and response capabilities, ensuring a more effective approach to managing and mitigating the effects of oil spills.

Technologies Utilized for DeepSpill Project







Language: Python

Framework: TensorFlow / Keras

Interface: Streamlit

Libraries: NumPy, Pandas, PIL, OpenCV, Matplotlib

Database: SQLite (for storing results)

Data Collection

Source: **Zenodo**

Title: Annotated RGB Images of Oil Spills in a Port Environment

Overview:

- Contains annotated RGB images of oil spills captured in a port environment.
- Dataset size: ~1.1 GB, split into train, validation, and test sets.

Purpose & Use:

- Designed for deep learning models to detect and localize oil spills in marine regions.
- Suitable for semantic segmentation, object detection, and environmental monitoring applications.

Significance:

- Enables AI-based analysis of marine pollution.

Data Loading & Quality Check

Objective:

Load and visualize a sample image and its mask to verify dataset integrity before model training.

Key Steps:

- Imported essential libraries: TensorFlow, NumPy, Matplotlib
- Loaded an image and its corresponding segmentation mask (grayscale)
- Normalized pixel values (0–1 range) for model readiness
- Visualized both side by side for quick inspection
- Verified image and mask properties — size, mode, and unique pixel values
- Thresholded the mask to binary values (0 for water, 1 for oil spill)

Outcome:

Ensured correct image–mask alignment, valid annotations, and binary segmentation format — crucial before preprocessing and training.

Image & Mask Preprocessing

Standardize and clean dataset inputs for reliable model training.

Resizing & Normalization:

- All images and masks resized to 256×256 pixels.
- Images normalized to range [0, 1] and converted to grayscale.
- Masks resized using nearest-neighbor interpolation to retain discrete labels.

Binary Conversion:

- Applied thresholding → 0 = water, 1 = oil spill.
- Ensured both image and mask tensors have consistent shape and dtype (float32).

Quality Verification:

- Visualized original vs resized samples side-by-side.
- Checked alignment, shape, and pixel intensity distributions for data integrity.

Dataset Loading & Preparation

Standardize and clean dataset inputs for reliable model training.

Resizing & Normalization:

- All images and masks resized to 256×256 pixels.
- Images normalized to range [0, 1] and converted to grayscale.
- Masks resized using nearest-neighbor interpolation to retain discrete labels.

Binary Conversion:

- Applied thresholding → 0 = water, 1 = oil spill.
- Ensured both image and mask tensors have consistent shape and dtype (float32).

Quality Verification:

- Visualized original vs resized samples side-by-side.
- Checked alignment, shape, and pixel intensity distributions for data integrity.

U-Net Model Architecture

Model Overview:

Implemented a U-Net CNN for oil spill segmentation.

Architecture Highlights:

- Encoder:
  - Sequential Conv → BatchNorm → ReLU → MaxPool blocks.
  - Extracts hierarchical spatial features.
- Bottleneck:
  - Deep feature representation layer with 16× filters.
- Decoder:
  - Transposed Convolutions for upsampling.
  - Skip Connections merge encoder features for precise localization.
- Output Layer:
  - 1×1 Conv with sigmoid activation → binary mask output (oil / non-oil).

Model Compilation & Custom Metrics

Objective:

To optimize the U-Net model using hybrid loss and custom performance metrics for segmentation accuracy.

Custom Loss Functions:

- Dice Loss: 1 - Dice Coefficient
- Combined Loss:
- Binary Crossentropy + Dice Loss for balanced optimization between pixel-wise accuracy and shape precision.

Purpose:

Improves segmentation of thin and irregular oil spill boundaries compared to BCE alone.

(Visual suggestion: show loss curve comparison – BCE vs. Combined Loss)

Training Configuration & Callbacks

Optimizer: Adam

Learning Rate Scheduling: ReduceLROnPlateau (reduces LR when validation loss plateaus)

Early Stopping: Stops training when val_loss stops improving.

Model Checkpoint: Saves the best model as unet_oilspill_best.keras.

Training Setup:

- Batch Size: 8
- Epochs: 50
- Validation Split: 20%
- Callbacks Used:
  - EarlyStopping
  - ModelCheckpoint
  - ReduceLROnPlateau

Optional Optimization:

Mixed precision training (tf.keras.mixed_precision) for faster computation on GPUs

Hyperparameter Tuning Overview

Objective:

Optimize U-Net model performance by fine-tuning key hyperparameters using Keras Tuner.

Why Tuning?

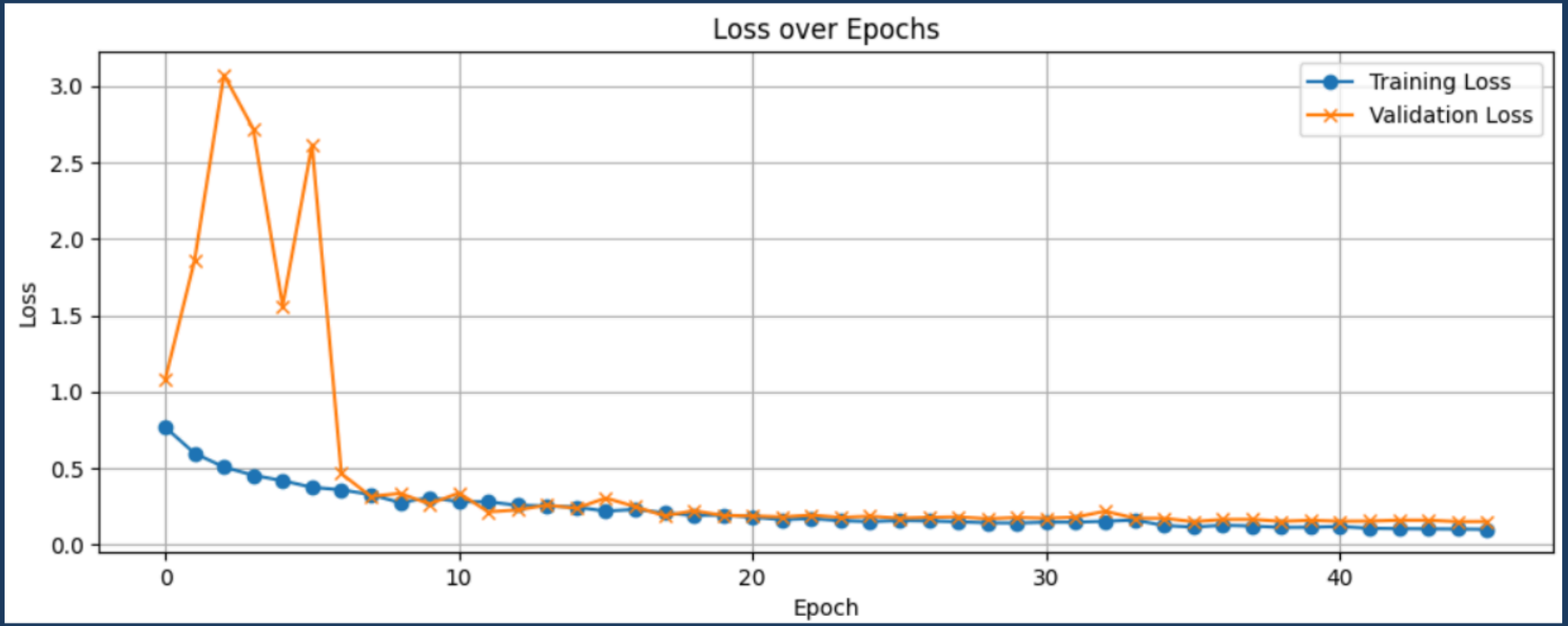
- Enhances model accuracy & IoU
- Reduces overfitting
- Improves convergence speed

Approach:

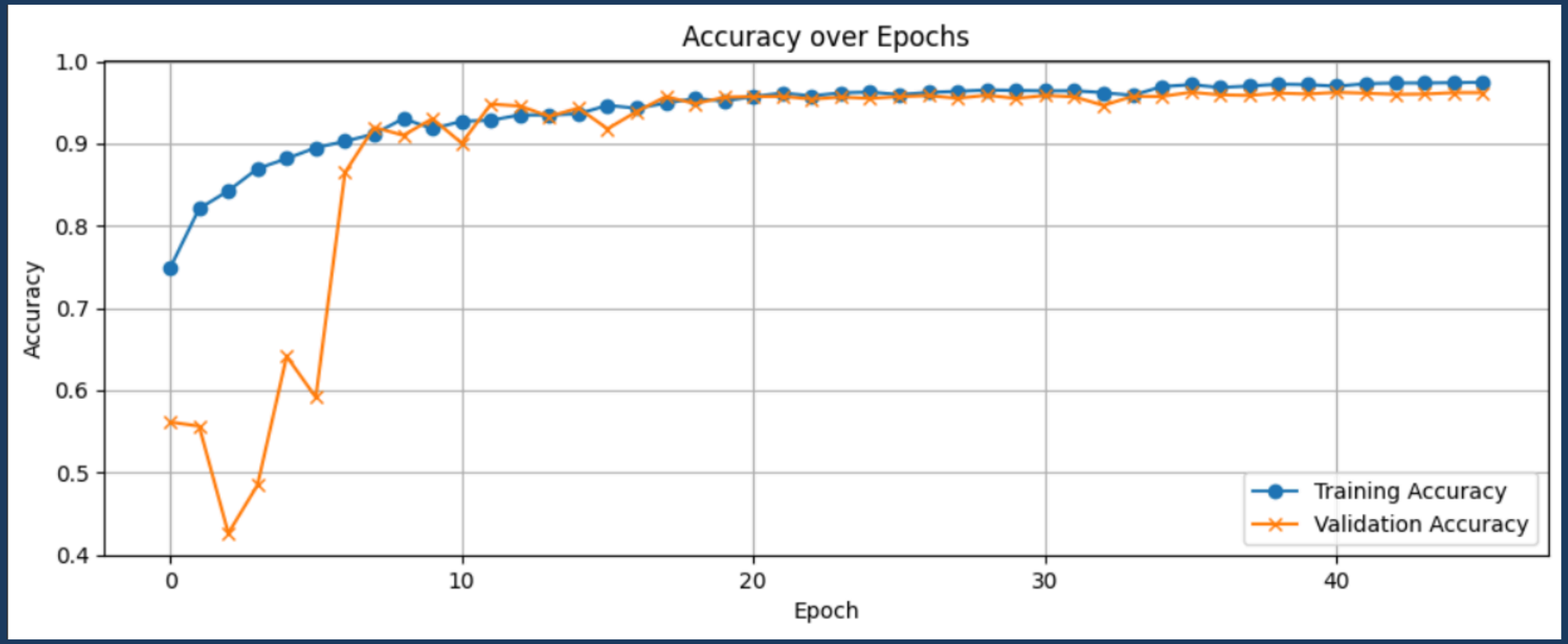
Used Random Search strategy for fast exploration with:

- max_trials = 5
- executions_per_trial = 1
- Subset of training data (30%) to reduce computation time

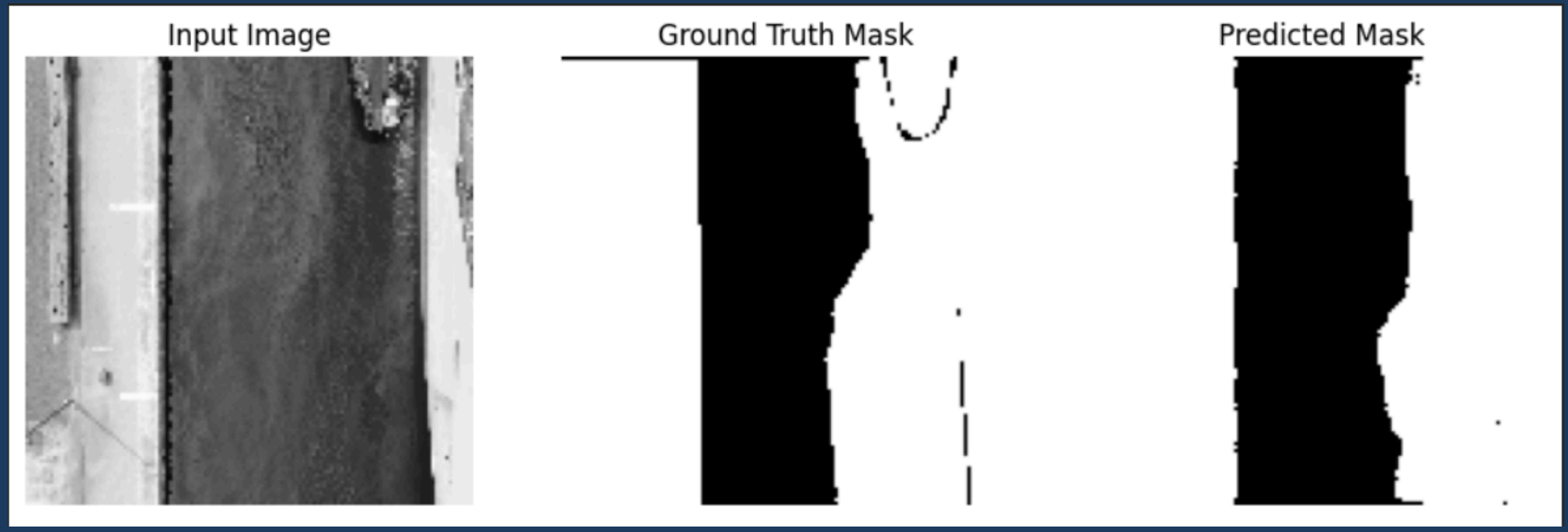
Model Evaluation & Performance Visualization



Model Evaluation & Performance Visualization



Prediction Visualization



Additional Visualizations & Evaluation

🌊 1. Automated Oil Spill Detection
Calculates spill ratio (predicted oil area %) from segmentation mask.
Decision Rule:
✅ OIL SPILL → if spill_ratio > 0.05
❌ NO OIL SPILL → otherwise
Enables real-time classification from segmentation outputs.

📊 2. Confusion Matrix Heatmap
Evaluates pixel-level classification accuracy.
Color-coded matrix of:
TP: Correct spill detection
TN: Correct background detection
FP/FN: False predictions
Helps verify balance between sensitivity and precision.